Ders 2 Genişletilmiş Notu: Hata Bulma ve Düzeltme, Sezgisel Sıkıştırma ve Run-Length Encoding

Ders 2 Genişletilmiş Notu: Hata Bulma ve Düzeltme, Sezgisel Sıkıştırma ve Run-Length Encoding

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Hata Nedir?
- 2. Hata Türleri
- 3. Hata Denetim Metotları: FEC ve ARQ
- 4. Tekrar ile Hata Sezme ve Düzeltme
- 5. Her İki Bit için 5 Bitlik Kod Kelimesi
- 6. Hamming Kodlama
- 7. Eşlik Sınaması
- 8. Günlük Hayattan Örnekler: UPC ve Posta Barkodu
- 9. CRC (Cyclic Redundancy Check)
- 10. MD5 ve SHA
- 11. Sezgisel Sıkıştırma: Braille ve Morse
- 12. Geri Dönüşümsüz Metin Sıkıştırma ve Sparse String
- 13. Baudot Kodlama
- 14. Önek Sıkıştırma (Front Compression)
- 15. Run-Length Encoding (RLE)
- 16. RLE'nin Temel Problemleri ve MNP5
- Örnek Sorular

1. Hata Nedir?

Önceki derste yalnız dersin terminolojisi konuşulmuştu. Bu derste dersin konu listesindeki ikinci başlık olan hata bulma ve düzeltmeye geçildi (slayt başlığı "Veri Sıkıştırma, Hata Bulma ve Düzeltme-2", ekrandan, 00:10:00). Hoca, elektronik kökenli öğrencilerin bildiği gibi sinyallerde oluşan hataların bulunması ve düzeltilmesi altında çok fazla konu olduğunu, kendisinin bunlardan algoritmaların içinde ve veri sıkıştırmada en çok kullanılan popüler olanlara yer verdiğini söyledi.

Hata, veriyi bir noktadan diğerine iletirken bitlerde oluşan bozulmadır. Ortama bağlı olarak oluşur: o sırada yıldırım düşmesi, şimşek çakması ya da kötü hava koşulları, işlem telli ya da telsiz hangi ortamda yapılıyorsa orada hataya yol açabilir.

Gönderilen verinin arkasına bazı kontrol bitleri konur ve bu bitler kontrol edilerek hata olup olmadığı anlaşılır. Hatalı bir durum varsa mesaj otomatik olarak kabul edilmez. Bazı durumlarda ise hatanın ne olduğu bulunabiliyorsa düzeltilir. Hangi durumda düzeltme yoluna gidildiği, hangisinde gidilmediği konunun ilerleyişinde görülecektir.

Veri içindeki bir bitlik bozulma tüm verinin yanlış anlaşılmasına yol açabilir. Gönderilen veri 0 ve 1'lerden oluşur. Bir bit 0'dan 1'e ya da 1'den 0'a dönerse verinin içeriği değişir ve karşı tarafa bozulmuş bir veri gider. Bu yüzden verinin bozulup bozulmadığının anlaşılması gerekir.

Hatayı sezmek mi, düzeltmek mi? Hatayı sezmek, düzeltmekten daha kolaydır. Çoğu zaman hata olduğunu anlarız ama düzeltemeyiz. Hatayı düzeltmek, veriyi bir kez daha göndermekten çok daha maliyetlidir; bu yüzden yeniden gönderim talep edilir.

Bir bitlik bozulmaları sezmek ve düzeltmek kolaydır. Ama örneğin 12 bitlik bir veri bloğunda 2, 3 ya da 4 bit bozulmuşsa bunu düzeltmek zordur, hatta düzeltilemez. Yine de bozulma olduğu anlaşılabilir ve karşı tarafa bildirilebilir.

2. Hata Türleri

Veri iletiminde iki çeşit hata vardır:

- **Burst error (patlama hatası):** Çevre koşulları nedeniyle alıcıya gerçek olmayan, anlamsız bilgiler gelir. Bilginin birçok yerinde bozulma olur ve gönderilen bilgi başka bir bilgiye dönüşür.
- **Random error (rastgele hata):** İletim yolundaki elektriksel gürültü nedeniyle bilgi bloğu içinde rastgele bir bit bozulur.

Tek bitlik hataya **spike**, birden fazla bitlik hataya **burst** denir. Bazen öyle bir durum oluşur ki bit ne 0'dan 1'e ne de 1'den 0'a dönmüştür; orada bir şey olmuştur ama nasıl bir bozulma olduğu anlaşılamaz. Oradaki bilgi tamamen silinmiştir; yanlış da olsa orada bir bilgi yoktur.

> **Ek bilgi (derste anlatılmadı):** Bilginin tamamen kaybolduğu bu hata türü kaynaklarda büyük olasılıkla "erasure" (silinme) hatası olarak geçer. Silinme hatasında bitin değeri bilinmez ama yeri bilinir.

3. Hata Denetim Metotları: FEC ve ARQ

Kitaplarda en çok kullanılan hata denetim metotları iki gruba ayrılır (ekrandan, 00:20:00):

1. FEC (Error Correction Code, Forward Error Correction, ileri yönlü hata denetimi)

- Hata denetimi için gönderilen veri kümesine ek bitler ekler.
- Hatayı bulur ve gerekirse alıcıda düzeltmeye çalışır.
- Hız kaybı yaşanmaz.
- Çok gürültülü ortamlarda kullanılamaz.
- Yeniden iletimin zor ya da imkânsız olduğu bağlantılarda ve veri kümesinin küçük olduğu uygulamalarda kullanılır.
- Örnekler: katlamalı kodlar, BCH kodları, Hamming kodları, Reed-Solomon kodları (slaytta "Reed-Salamon").

Hoca bunu şöyle açıkladı: elimdeki veriyi karşı tarafa gönderirken verinin sonuna, kullandığım algoritmaya bağlı olarak 1, 2, 3, 4 ya da daha fazla ek bit eklerim. Bu ek bitlerde hiç bozulma olmadığı kabul edilir. Alıcı bu ek bitler üzerinden yaptığı işlemlerle verinin bozulmadan gelip gelmediğini kontrol eder, hatayı bulur, düzeltebiliyorsa düzeltir. Bir kontrol yapıldığı için önemli bir hız kaybı yoktur, belki biraz zaman kaybı olabilir. Ama ileri yönlü hata denetimi, yani Hamming kodlarının kullanılacağı ortamlar çok gürültülü ortamlar olmamalıdır. Gönderilen verilerin oluşturduğu veri kümesi bazen küçüktür; o zaman Hamming kodu tercih edilir. Hoca, Hamming kodunu en önemli hata belirleme yöntemlerinden biri olduğu için ayrıntılı anlatacağını söyledi.

2. ARQ (Automatic Repeat Request, otomatik tekrar isteği)

- Hatayı bulma ve bozulan verinin yeniden iletilmesi için alıcı taraftan istekte bulunulması işlemidir.
- Uygulamalarda bu teknik kullanılır, çünkü aynı hatayı tespit için gerekli bit sayısı, düzeltmek için kullanılan bit sayısından çok daha azdır.
- Örnekler: CRC kodları, seri eşlik (parity), blok eşlik, modül toplamı.

ARQ'da hata oluşmuşsa bulunur, ama bulunduğu an düzeltilmeye çalışılmaz. Alıcı göndericiye "bana gönderdiğin veride bir bozulma oldu, bu veriyi tekrar gönder" der. Yani FEC'de hata, geldiği yerde düzeltilmeye çalışılır; ARQ'da düzeltmeyle uğraşılmaz, verinin yeniden gönderilmesi istenir. Veriyi yeniden istemek, hatayı düzeltmekten çok daha kolay ve az maliyetlidir.

Eşlik bitleri (tek eşlik, çift eşlik) ve CRC kodları bu gruptadır; hoca CRC'nin de en önemlilerden biri olduğunu ve ayrıntılı anlatacağını söyledi.

4. Tekrar ile Hata Sezme ve Düzeltme

Hoca en basit yöntemden başladı (ekrandan, 00:24:00).

4.1 Kontrol Bitsiz Gönderim

A noktasından B noktasına `1011010100110...` verisi gönderilsin. Alıcıya dördüncü bit bozulmuş olarak `1010010100110...` gelsin. Alıcı bunun bozulduğunu anlamaz; orijinalinin 1 olduğunu nereden bilsin? Veriyi alır ve kullanır. Bu veri sıkıştırılıp gönderilmiş olsaydı, açıldığında orijinal veri elde edilemezdi; çünkü bir bitlik bozulma fark edilemedi. Slayttaki sonuç: "Hata sezilemez."

4.2 Her Biti Çift Sayıda Yazmak

Problemi çözmek için her bit iki kez gönderilir: 1 yerine `11`, 0 yerine `00`. Gönderilen veri `11 00 11 11 00 11 00 00 11 11 00 ...` olur. Yine bir bitlik bozulma olsun ve alıcıya `11 00 11 01 00 ...` gelsin.

Alıcı bir hata olduğunu anlar, çünkü kontrol için eklenen bitin, bitin kendisinin aynısı olması gerektiğini bilir. Burada farklı iki bit gelmiştir. Ama düzeltebilir mi? Önceden 1 olan mı 0 oldu, yoksa 0 olan mı 1 oldu? Bunu bilemez. Hatayı kendisi düzeltemese de karşı tarafa "bana gelen bilgi hatalı, yeniden gönder" der. Slayttaki sonuç: "Hatayı sezeriz ama düzeltemeyiz."

İki bitlik bozulma olsaydı, yani `11` yerine `00` gelseydi, bozulma anlaşılmazdı; bu orijinal veri sanılırdı. Bu yöntem yalnız bir bitlik bozulmaları tespit eder ve veriyi iki katı yer harcayarak gönderir.

4.3 Her Biti İki Kopya ile Yazmak

Bu kez her bitin iki kopyası eklenir: 1 yerine `111`, 0 yerine `000`. Gönderilen veri `111 000 111 111 000 111 000 000 111 111 000 ...` olsun; alıcıya dördüncü grup `011` olarak gelsin (slaytta son grup da `010` olarak bozulmuş gösterilmiştir).

Alıcı her bitin iki kopyasıyla geleceğini bilir; burada öyle gelmemiştir, yani hata olduğunu anlar. Bozulma bir bitlikse düzeltebilir: burada iki tane 1 vardır, üç tane 1 olması gerekir, o hâlde 0 yerine 1 konur. İki bit değişmişse, örneğin `000` grubu `011` olmuşsa, yine bozulma olduğu bilinir ama düzeltilemez; asıl grubun `111` mi `000` mı olduğu bilinmez.

Hoca şimdiye kadar yapılanları şöyle özetledi:

- Biti bir kez gönderdim: hatayı bile göremiyorum.
- Biti iki katına çıkardım: hatayı buluyorum ama düzeltemiyorum.
- Biti üç katına çıkardım: hatayı buluyorum, bir bitlikse düzeltiyorum, birden fazlasını düzeltemiyorum.

Bunların hepsi bir bitlik hatayı düzeltmek amacıyla ortaya atılmış uygulamalardır. Hata bulunup düzeltilemiyorsa karşı taraftan yeniden gönderim istenir.

5. Her İki Bit için 5 Bitlik Kod Kelimesi

Yöntem bir adım ileri taşınır: bu kez iki bitlik veriler kodlanır (ekrandan, 00:30:00). İki bit varsa $2^2 = 4$ farklı durum vardır: `00`, `01`, `10`, `11`. Her iki bit için 5 bitlik bir kod kelimesi oluşturulur:

Mesaj	Kod kelimesi
00	00001
01	01010
10	10100
11	11111

2 bit yerine 5 bit kullanıldığı için maliyet 2,5 kattır. Bu kod kelimeleri rastgele seçilmemiştir: kod kelimeleri en az 3 bitte değişiklik olacak şekilde seçilmiştir. Örneğin birinci ile ikinci kod kelimesi arasında 3 bit fark vardır; birinci ile dördüncü arasında 4 bit fark vardır.

Bir kod kelimesinde yalnız bir bit değişirse karşıya çıkabilecek bütün durumlar slayttaki tabloda verilmiştir (ekrandan, 00:30:00):

Bozulmamış	00001	01010	10100	11111
1. bit bozuk	10001	11010	00100	01111
2. bit bozuk	01001	00010	11100	10111
3. bit bozuk	00101	01110	10000	11011
4. bit bozuk	00011	01000	10110	11101
5. bit bozuk	00000	01011	10101	11110

Kod kelimeleri birbirinden en az 3 bit farklı olduğu için tek bit bozulmuş bir kelime yalnız bir kod kelimesine yakındır. Alıcı gelen kelime tabloda yoksa onun hangi kod kelimesinin bozulmuş hâli olduğuna bakar ve orijinali bulur. Çözümlü örnek "Örnek Sorular" bölümündedir.

Hoca karşılaştırmayı şöyle yaptı: üç kopya yönteminde bir bitlik hata düzeltilir ama 1 bit yerine 3 bit harcanır; bu yöntemde de bir bitlik hata düzeltilir ama 1 bit yerine 2,5 bit harcanır. Amaç veri sıkıştırma ise hatayı bulmak için veriye eklenen ekstra bit sayısını da azaltmak gerekir; bu yüzden bu yöntem daha idealdir.

Soru: Bu yöntemde bir bitlik bozulma olduğunu varsayarak ilerliyoruz sanırım. 2 bit bozulursa, örneğin ilk kod kelimesi aslında `00`'a da karşılık gelebilir mi? **Cevap:** 2 bitlik bozulmalar bu yöntemle tespit edilemez; anlatıma da bu şekilde girildi. 2 bitlik bozulma varsa bu tablo işe yaramaz. Bir dosya gönderildiğini söylesek de arka planda dosyanın bitleri belki 8'er bit 8'er bit gider; bir milyon bit aynı anda gönderilmez. Veri küçük parçalar hâlinde aktarıldığı için bir bitlik bozulma olabileceği varsayılır. Bir bitten fazla bozulma olursa alıcı zaten anlamsız bir şey göreceği için göndericiden veriyi yeniden ister.

6. Hamming Kodlama

Hoca Hamming kodlamayı "gerçekten önemli bir kodlama" diye tanıttı.

> **Hocanın vurgusu:** Hamming kodlamada da tek bitlik bozulma bulunur ve düzeltilir, ama 1 bit yerine yalnız 1,75 bit kullanılır. Önceki yöntem aynı işi 2,5 bitle yapıyordu.

6.1 Kodlama

Slayttaki tanım (ekrandan, 00:40:00): Hamming kodlama, 4 bitlik verideki yer değiştirmeler ile 7 bitlik kod kelimelerinin elde edildiği lineer bir kodlamadır (1 bit ≈ 1,75 bit). Bütün kod kelimelerinde en az 3 bitte değişim vardır; bir bitte oluşan bozulmalar düzeltilebilir.

4 bitlik mesaj $b_1b_2b_3b_4$ olsun. İlk dört bit mesajın aynısıdır; arkasına 3 kontrol biti eklenir:

$$b_1b_2b_3b_4\ (b_1 \oplus b_2 \oplus b_3)\ (b_1 \oplus b_3 \oplus b_4)\ (b_2 \oplus b_3 \oplus b_4)$$

Yani $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_3 \oplus b_4$, $b_7 = b_2 \oplus b_3 \oplus b_4$.

$\oplus$ işlemi slaytta "parity addition" olarak geçer (ekrandan, 00:46:30): `01` ya da `10` → `1`, `00` ya da `11` → `0`. Yani girişler aynıysa çıkış 0, farklıysa 1'dir. Hoca buna XOR (exor) da denebileceğini söyledi.

> **Ders sonrası not:** Hoca anlatırken bir yerde "bitwise or ile egzorluyor" dedi (00:40:06). Burada kullanılan işlem OR değil, XOR'dur (parity addition): `11` girişi OR'da 1, XOR'da 0 verir.

Slayttaki kod tablosu (ekrandan, 00:40:00; slaytta `1111` satırı yer almıyor):

Mesaj	Kod kelimesi	Mesaj	Kod kelimesi
0000	0000000	0110	0110010
0001	0001011	0101	0101110
0010	0010111	0011	0011100
0100	0100101	1110	1110100
1000	1000110	1101	1101000
1100	1100011	1011	1011010
1010	1010001	0111	0111001
1001	1001101		

Tablodaki bütün kod kelimeleri kurala uygundur ve birbirinden en az 3 bit farklıdır (python ile doğrulandı).

Kodlama örneği (00:40:32): Mesaj `1100` olsun: $b_1 = 1$, $b_2 = 1$, $b_3 = 0$, $b_4 = 0$.

- $b_5 = b_1 \oplus b_2 \oplus b_3 = 1 \oplus 1 \oplus 0$. $1 \oplus 1 = 0$ (girişler aynı), $0 \oplus 0 = 0$. $b_5 = 0$.
- $b_6 = b_1 \oplus b_3 \oplus b_4 = 1 \oplus 0 \oplus 0$. $1 \oplus 0 = 1$, $1 \oplus 0 = 1$. $b_6 = 1$.
- $b_7 = b_2 \oplus b_3 \oplus b_4 = 1 \oplus 0 \oplus 0 = 1$.

Gönderilecek kod kelimesi `1100011`'dir; tablodaki satırla aynıdır. Girdi 4'er bitlik parçalara bölünür ve her 4 bit için karşılığı olan kod kelimesi gönderilir. Hoca, bunu tahtada tebeşirle anlatmanın çok daha kolay olduğunu, ekranda fareyle yazmanın zor olduğunu söyledi.

6.2 Alıcıda Kontrol

Alıcı gelen kelimede bozulma olup olmadığını üç şartla kontrol eder (ekrandan, 00:46:30):

$$b_1 \oplus b_2 \oplus b_3 \oplus b_5 = 0, \qquad b_1 \oplus b_3 \oplus b_4 \oplus b_6 = 0, \qquad b_2 \oplus b_3 \oplus b_4 \oplus b_7 = 0$$

Üç şart da sağlanıyorsa bir bitlik bozulma yoktur.

Bozulmuş kelime örneği (ekrandan, 00:46:30): Alıcıya `1100100` gelsin. Hoca üç şartı hesapladı (slayttaki yeşil yazılar):

- $b_1 \oplus b_2 \oplus b_3 \oplus b_5 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$
- $b_1 \oplus b_3 \oplus b_4 \oplus b_6 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$
- $b_2 \oplus b_3 \oplus b_4 \oplus b_7 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$

Şart sağlanmadı; bozulma olduğu bellidir. Burada üçünde de 1 çıktı. Hoca her zaman üçünde 1 çıkmayacağını, bazen yalnız birinde, bazen birinci ile ikincide, bazen ikinci ile üçüncüde 1 çıkabileceğini söyledi. Yalnız üçü de 0 çıktığında hiçbir bozulma yoktur.

6.3 Bozulan Bitin Bulunması

Bozulmuş kod kelimesinin bitleri $\tilde{b}_1\tilde{b}_2 \ldots \tilde{b}_7$ ile gösterilir (tilde, bozulmuş olabileceğini belirtir). Slayttaki parity check kontrolü (ekrandan, 00:50:00):

$$P_1 = \tilde{b}_1 \oplus \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_5, \quad P_2 = \tilde{b}_1 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_6, \quad P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_7$$

- $P_1 = P_2 = P_3 = 0$ ise kelime doğrudur. İlk dört bit mesajdır; dördüncü bitten sağa doğru $\tilde{b}_5\tilde{b}_6\tilde{b}_7$ atılır.
- P_j'lerden biri 1, diğer ikisi 0 ise bozulan bitin hangi P_j'den geldiği bulunur. İlgili P_j'de olup diğerlerinde olmayan bit seçilir ve tersi alınır.

Hocanın anlattığı örnekler:

- **Yalnız $P_2 = 1$:** P_2'de olup P_1 ve P_3'te olmayan bit aranır. b_1 bozulmuş olsaydı P_1 de hatalı çıkardı; b_3 bozulmuş olsaydı P_1 ve P_3 de hatalı çıkardı; b_4 bozulmuş olsaydı P_3 de hatalı çıkardı. Diğerlerinde bulunmayan bit b_6'dır. Bozulan b_6'dır; değeri tersine çevrilir.

- $P_1 = P_2 = 1$**,** $P_3 = 0$**:** Hem P_1 hem P_2'de olup P_3'te olmayan bit yalnız b_1'dir. b_1 bozulduğu için P_1 ve P_2 1 vermiştir; P_3'te b_1 geçmediği için orada bozulma görünmez. b_1 ters çevrilir (slaytta da bu örnek yazılı, ekrandan, 00:50:00).
- $P_1 = P_2 = P_3 = 1$**:** Üçünü de bozan bit, üçünde de bulunan b_3'tür. b_3 ters çevrilince doğru mesaj bulunur ve kontroller tekrar yapılınca hepsinin 0'a düştüğü görülür.

Önceki örnekteki `1100100` kelimesinde üç kontrol de 1 olduğu için b_3 ters çevrilir: `1110100`. Bu, tablodaki `1110` mesajının kod kelimesidir (çözüm "Örnek Sorular" bölümünde).

Soru: Bu durumda b_5, b_6, b_7 aynı anda bozulmuş olabilir mi? **Cevap:** Burada yalnız bir bitlik bozulmalar bulunup düzeltiliyor, bir bitten fazla olanlar değil. Hamming koduyla 2 bitlik bozulma düzeltilemez; bazen tesadüfen düzeltilebilir, bazen düzeltilemez. Ama 1 bitlik bozulma %100 garantili olarak düzeltilir. 2 ya da 3 bit bozulursa bu burst error olur; bunlarda düzeltme yapılmaz, veri yeniden istenir. Şu an spike hataları, yani bir bitlik bozulmalar ele alınıyor.

7. Eşlik Sınaması

Bu yöntemde hata sezilir ama düzeltilemez; karşı taraftan veri yeniden istenir. Slayttaki maddeler (ekrandan, 00:56:30):

- Gönderilen veride oluşan tek sayıdaki hatayı sezmek için kullanılır.
- Verideki birlerin sayısını tek ya da çift olarak düzenlemektir.
- Eşlik biti 1 ya da 0 yapılarak tüm veri grubunun içindeki birlerin sayısının çift ya da tek olması sağlanır.
- Tek eşlik yönteminde veri içindeki 1'lerin sayısı tek ise eşlik biti 0 olur.
- Çift eşlik yönteminde veri içindeki 1'lerin sayısı tek ise eşlik biti 1 olur.

Slayttaki örnekte veri `0100010100`'dür; birlerin sayısı $1 + 1 + 1 = 3$, yani tektir. Tek eşlikte eşlik biti 0 olur ve gönderilen `01000101000` olur. Çift eşlikte eşlik biti 1 olur ve gönderilen `01000101001` olur. Hoca, tek ya da çift eşlik seçmenin başka bir anlamı olmadığını söyledi.

Bitlerden biri bozulsun, örneğin bir 1, 0 olsun. Birlerin sayısı çift olur; tek eşlikte tek sayıda 1 olması gerekirken çift sayıda geldiği için bozulma olduğu söylenebilir. Ama iki bit bozulursa, örneğin iki 0, 1 olursa, birlerin sayısı 5 olur; 5 de tektir. İki bozulma olmasına karşın hiç bozulma yokmuş gibi algılanır. Bu yöntem yalnız tek sayıdaki bozulmaları söyler ve düzeltme yapmaz.

> **Hocanın vurgusu:** En çok tercih edilen ve kullanılan Hamming kodlamadır: hem daha az bit kullanılır (1 bit yerine 1,75 bit), hem bir bitlik hata bulunur, hem de bulunan hata %100 doğru olarak düzeltilir.

8. Günlük Hayattan Örnekler: UPC ve Posta Barkodu

8.1 Universal Product Code (UPC)

Ürünlerin üzerindeki barkodda ince, kalın ve orta kalınlıkta çizgiler vardır ve bunlar belirli bitlere karşılık gelir (ekrandan, 01:00:00):

- ince siyah çizgi = `1`, biraz kalın siyah çizgi = `11`, kalın siyah çizgi = `111`, çok kalın siyah çizgi = `1111`
- ince boşluk = `0`, biraz geniş boşluk = `00`, geniş boşluk = `000`, çok geniş boşluk = `0000`

Barkod okutulduğunda çizgilerin her birinin ondalık karşılıkları okunur. Kodun sol ve sağ tarafı vardır; her dijit 7 ikili dijitle gösterilir ve ortada bir ayıraç (separatör) bulunur. Slayttaki notlar (ekrandan, 01:20:00):

- Sol taraftaki her 7'li grup 0 ile başlar, 1 ile biter; sağ taraftaki her 7'li grup 1 ile başlar, 0 ile biter.
- Sol taraftaki her 7'li grupta birlerin sayısı tektir (örneğin 3 ya da 5); sağ taraftakilerde çifttir (örneğin 4).

Slayttaki kodlama tablosu (ekrandan, 01:20:00):

Dijit	Sol	Sağ
0	0001101	1110010
1	0011001	1100110
2	0010011	1101100
3	0111101	1000010
4	0100011	1011100
5	0110001	1001110
6	0101111	1010000
7	0111011	1000100
8	0110111	1001000
9	0001011	1110100

UPC kodu $1 + 5 + 5 + 1$ dijittir: ilk dijit ürünün ne tip ürün olduğunu, sonraki 5 dijit üreticinin kodunu, sonraki 5 dijit ürünün kodunu gösterir; son dijit kontrol dijitidir. Kontrol dijiti barkodda bulunur ama ondalık değeri rakamla yazılmaz. Okuyucu, çizgileri okuyarak bunun hangi koda karşılık geldiğini bilir.

Kontrol dijitinin hesabı (ekrandan, 01:20:00): İlk 11 dijit alınır:

$$3a_1 + a_2 + 3a_3 + a_4 + 3a_5 + a_6 + 3a_7 + a_8 + 3a_9 + a_{10} + 3a_{11} \equiv A \pmod{10}$$

Kontrol dijiti C (ya da a_{12}) ise $C = 10 - A$'dır. Katsayılar 3, 1, 3, 1, ... diye gider. Çözümlü örnek "Örnek Sorular" bölümündedir; örnekteki ürünün kontrol dijiti 4'tür.

Bir dijit hatalı okunursa aynı işlem kontrol dijitini vermez; okuyucu barkodun bir dijitinde hata olduğunu anlar. Markette bir ürün okutulduğunda "ürün tanınamadı" uyarısı, barkodun doğru okunamamasından (yanlış tutma, silik basım) ya da ürünün marketin barkod listesinde olmamasından kaynaklanabilir. Hoca, üreticinin farklı bir polinom kullanabileceğini, katsayıları farklı verebileceğini ya da başka bir sayıya göre mod alabileceğini; kontrol dijiti nasıl oluşturulmuşsa onu okuyacak programın da ona göre yazılacağını söyledi.

8.2 ABD Posta Sistemi Barkodu

ABD'de posta sisteminde kullanılan bir uygulamada da kısa ve uzun çizgilerden oluşan kodlar vardır (ekrandan, 01:25:40). Slayttaki kaynakta baştaki ve sondaki uzun çubuklar koruma çizgisidir (guard lines) ve her beş çizgilik grup bir dijiti gösterir. Slaytta tablo şöyledir:

Dijit	İkili kod	Dijit	İkili kod
1	00011	6	01100
2	00101	7	10001
3	00110	8	10010
4	01001	9	10100
5	01010	0	11000

Slayttaki açıklama: her grup 2 uzun, 3 kısa çubuktan oluşur. Kısa yerine uzun ya da tersi okunduğunda hata oluşur. Oluşan bir bitlik hata düzeltilebilir mi?

Burada da son dijit kontrol dijitidir: bütün dijitler toplanıp 10'a göre mod alındığında sonuç kontrolü sağlamalıdır. Okunan sayı `2007755761`'dir ve son 1 kontrol dijitidir. Hoca beşinci dijitin (7) okunamadığını varsaydı; çözüm "Örnek Sorular" bölümündedir.

Hoca iki örneği şöyle bağladı: bir kodlamada ince-kalın siyah-beyaz çizgiler, diğerinde uzun-kısa çizgiler vardır. Hepsi 0 ile 9 arasında bir dijite karşılık gelir; ürünlere bu dijitlerden kodlar verilir, ama ürünün üzerinde dijitler değil çizgiler bulunur. Okunan kodun hatalı olup olmadığını anlamak için sonuna bir kontrol (parity) kodu eklenir. Bu kod belirli bir işlemle elde edilir; kontrol sırasında aynı işlem yapılır. Aynı sonuç elde edilemiyorsa kod hatalıdır.

9. CRC (Cyclic Redundancy Check)

Kodlar iki türlüydü: veri kısaysa hata bulunur ve düzeltilmeye çalışılır (Hamming); veri daha uzunsa bozulma olduğunda düzeltmek yerine mesajın yeniden gönderilmesini istemek daha kolaydır ve zamandan tasarruf sağlar. Bu ikinci türün örneği CRC'dir.

Hoca adında geçen "redundancy" kavramını hatırlattı: redundancy artıklık demekti. Sistemin entropisi, bir sembolü kodlamak için ortalama kaç bite ihtiyaç olduğunu söyler; kullanılan uygulama ise sembole belli sayıda bit verir. Artıklık bu ikisi arasındaki farktır. Fark sıfıra ne kadar yakınsa ideal bir yöntem seçilmiştir; ne kadar büyükse yöntem o kadar kötüdür ve veri başka yöntemlerle daha iyi sıkıştırılabilir.

9.1 CRC Kodunun Oluşturulması

Slayttaki örnek (ekrandan, 01:34:50):

- Gönderilecek mesaj: `0111101`.
- Üreteç fonksiyonu: $G(x) = x^3 + x^2 + 1$. Bunu belirleyen kişidir.
- Üreteç fonksiyonunun derecesi CRC kodunun bit uzunluğunu verir ve başlangıçta bu bitlerin değeri 0'dır. Derece 3 olduğu için CRC kodu 3 bitliktir; 5. dereceden bir fonksiyon olsaydı 5 bit eklenirdi.
- $G(x)$'in katsayıları kontrol dizisini verir: x^3'ün katsayısı 1, x^2'nin 1, x terimi olmadığı için katsayısı 0, x^0'ın katsayısı 1. Yani $G =$ `1101`.

İşlem adımları:

1. Mesajın arkasına 3 tane 0 eklenir: `0111101000`.
2. Mesajın ilk dört biti `1101` ile XOR'lanır (girişler aynıysa 0, farklıysa 1).
3. Soldaki bit iptal edilir.
4. Sıradaki ilk bite bakılır: 0 ise üreteç yerine `0000`, 1 ise üretecin kendisi (`1101`) yazılır ve yeniden XOR'lanır.
5. 3 bit kalıncaya kadar devam edilir. Kalan 3 bit CRC kodudur ve orijinal mesajın arkasına eklenerek gönderilir.

Slayttaki el yazısı işlemin satırları şöyledir:

```
0111101 000      mesaj + CRC yeri
1101             XOR
 010101000       soldaki bit atıldı
 0000            ilk bit 0 -> 0000
  10101000
  1101           ilk bit 1 -> 1101
   1111000
   1101
    010000
    0000
     10000
     1101
      1010
      1101
       111       CRC kodu
```

Sonuçta CRC kodu `111` olur ve `0111101111` gönderilir.

> **Ders sonrası not:** Slayttaki işlemde ilk adımda mesajın ilk biti 0 olduğu hâlde `1101` ile XOR yapılmıştır; oysa sonraki adımlar için konan kural (baştaki bit 0 ise `0000`) ilk adımda da geçerlidir. Standart CRC'de (mod 2 polinom bölmesi) `0111101000`'in `1101`'e bölümünden kalan `011`'dir; gönderilen `0111101011` olur ve alıcıda kalan `000` çıkar (python ile doğrulandı). Slayttaki yol gönderen ve alan aynı işlemi yaptığı için kendi içinde tutarlı sonuç verir, ama standart CRC değeri `011`'dir.

9.2 Alıcıda Kontrol

Alıcı CRC eklenmiş kodu alır ve aynı işlemi yapar (ekrandan, 01:40:00). Slaytta alınan `0111101111` için işlem sonunda `0000` elde edilir: "Bozulma YOK". Alıcı orijinal mesajı alıp kullanır.

Slayttaki ikinci soru: "Üçüncü bit hatalı alınmış olsaydı, anlaşılacak mıydı?" Alınan veri `0101101111` olsun. Aynı işlem yapılınca sonuç `0001` çıkar. Sıfır elde edilmesi gerekiyordu; elde edilemediği için bir bozulma vardır. Ama bozulmanın nerede olduğunu bulmanın imkânı yoktur. Bunu bulmak için harcanacak emek, zaman ve bit yerine karşı tarafa "bana gönderdiğin veriyi tekrar gönder" mesajı gönderilir.

Hem gönderen hem alan üreteç fonksiyonunu bilmelidir; yoksa çözme sırasında bu fonksiyon bilinemez. Kodlayan da çözen de algoritma üreteç kodunu bildiği için çözme yapılabilir.

Hoca bu bölümü şöyle bağladı: bir bitlik bozulmalara spike, çok daha büyük bozulmalara burst denir. Burst durumlarda verinin yeniden gönderilmesi istenir; spike'larda ise bozulma görüldüyse düzeltilebilir. Hamming kodları daha kısa verilerin gönderilmesinde tercih edilirken CRC kodları daha uzun verilerde kullanılabilir; ikisinin de farklı uygulama alanları vardır.

10. MD5 ve SHA

Bu konunun son slaytı MD5'tir (ekrandan, 01:42:30). Hoca bunun günlük hayatta çok kullanıldığını, ama çoğu zaman adının bu olduğunu bilmediğimizi söyledi. "Digest" özet demektir. Slayttaki maddeler:

- Bir internet standardıdır; veri güvenliğinde de kullanılır.
- Veri güvenliği ve bütünlüğünde kullanılır.
- Büyük bir veriden küçük bir özet oluşturulur ve bu özet bilgiden orijinal veriye dönülür.
- Tam güvenli bir algoritma değildir, ama hızlı çalışan bir algoritma olduğu için tercih edilir.
- İstenen her boyuttaki veriden 128 bitlik bir özet elde eder. Bu özet bilgiyi de 32 bitlik 4 eşit parçaya böler.
- MD5'in sıkıştırma fonksiyonunda bir çakışma bulunmasından yola çıkarak SHA (Secure Hashing Algorithm) gibi algoritmalara yönelinse de MD5'in kırılabilirliği ispat edilmediğinden günümüzde hâlâ veri güvenliği için tercih edilir.
- SHA1, $(2^{64} - 1)$ bitlik mesajdan 160 bitlik özetleme değeri (hash fonksiyonu) üretir.
- MD5 tek yönlü bir algoritmadır, geri dönüşümü yoktur. Örneğin kullanıcı şifresini unutursa sistem şifresini veremez, yeniden şifre oluşturması istenir.
- Eldeki veri 512 bitlik bloklara ayrılır ve her birine aynı işlem uygulanır. Veri 512'nin katı değilse padding işlemi uygulanır.

Hoca, MD5'in hash fonksiyonları gibi düşünülmesini istedi. Lisanstaki veri yapıları dersinden hash'leri hatırlattı: bir veriyi tabloya yerleştirirken hangi sıraya yerleştirileceğini belirlemek ve veriye bir ya da iki erişimle ulaşmak için veri bir hash fonksiyonundan geçirilir; sonuç neyse veri tablonun o sırasına yazılır, o sıra doluysa ikinci bir hash fonksiyonu ya da başka işlemler kullanılır. MD5'in tam güvenli olmamasının nedeni de buna benzer bir durumdur; hoca bunun çok nadir bir durum olduğunu ve aynı kişinin başına aynı platformda gelme olasılığının çok zayıf olduğunu söyledi. Bunun yerine SHA ortaya atılmıştır; MD5 128 bitlik özet üretirken SHA 160 bitlik özet üretir. Hoca SHA'nın çok fazla kullanılmadığını söyledi.

Şifre saklama örneği: MD5 çok hızlı ve tek yönlü olduğu için kullanıcı şifreleri bir yerde saklanırken MD5'ten geçirilerek saklanır. Şifre veritabanında açık hâliyle değil, 128 bitlik özet olarak tutulur. Özete bakan kişi orijinal şifreyi bilemez. Kullanıcı her şifre girdiğinde o şifrenin özeti oluşturulur ve saklı özetle karşılaştırılır. "Şifremi unuttum" denince sistem "şifren budur" demez, çünkü şifreyi bilmez; şifrenin değiştirilmesini ister. "Şifremi unuttum" deyince şifre e-postayla gönderiliyorsa bu, şifrenin veritabanında şifrelenmeden saklandığını ve hiçbir güvenliği olmadığını gösterir. MD5 ise geri dönüşsüzdür; özete bakarak orijinal hâle dönülemez. Hoca, MD5'in nasıl çalıştığının bilinmediğini, bilinse geri dönüşünün de yapılabileceğini ekledi.

> **Ders sonrası not:** Slaytta "bu özet bilgiden orijinal veriye dönülür" yazıyor; ama aynı slayt ve hoca MD5'in tek yönlü olduğunu, geri dönüşü olmadığını söylüyor. Doğ-

rusu: özetten orijinal veriye dönülemez. Özet, veri değişti mi diye kontrol etmek (bütünlük) için kullanılır.

Ders sonrası not: Slaytta MD5'in kırılabilirliğinin ispat edilmediği ve hâlâ veri güvenliği için tercih edildiği yazıyor. MD5'te çakışma üretmek büyük olasılıkla 2004-2005'ten beri pratikte mümkündür ve MD5 bugün güvenlik amacıyla önerilmez; SHA-1 için de büyük olasılıkla 2017'de pratik çakışma gösterilmiştir. Hocanın "MD5'in nasıl çalıştığını bilmiyoruz" sözü de algoritmanın kendisi için geçerli değildir; MD5'in adımları büyük olasılıkla açık bir standartta (RFC 1321) yayımlanmıştır. Geri dönülemezliği algoritmanın gizliliğinden değil, özet fonksiyonunun tek yönlü yapısından gelir.

Ek bilgi (derste anlatılmadı): Şifre saklamada bugün düz MD5 yerine her kullanıcıya özel rastgele bir ek (salt) ile birlikte bilerek yavaş çalışan fonksiyonlar (örneğin bcrypt, scrypt, Argon2) önerilir. Hızlı bir özet fonksiyonu, saldırganın çok sayıda şifre denemesini kolaylaştırır.

11. Sezgisel Sıkıştırma: Braille ve Morse

Bu noktada yeni bölüme, sıkıştırma algoritmalarına geçildi. Hoca şimdiye kadar sıkıştırmayla ilgili hiçbir şey görülmediğini, önce en temel tekniklerin görüleceğini söyledi. Kullanılan diğer yöntemler bu temel teknikler üzerine kurulur ya da bunların birleştirilmesiyle (hibrit) ortaya çıkar.

Hoca bir kitapta okuduğu bir rivayeti anlattı: Roma döneminde bir ülkeden diğerine bilgi gönderilecektir. Elçi mesajı üzerinde taşırsa çalınabilir. Elçinin kafası kazıtılır, mesaj kafa derisine yazılır, saçın uzaması beklenir ve elçi öteki ülkeye gönderilir. Yolda sorsalar bile söyleyemez, mesaj kafasında yazılıdır. Karşı tarafta saç yeniden tıraş edilir ve bilgi okunur. Hoca, bunun aslında bir sıkıştırma değil, kafa derisine bir kodlama olduğunu söyledi.

Sezgisel sıkıştırma (intuitive compression) algoritmalarının ilk örnekleri Braille ve Morse kodlamadır (ekrandan, 01:50:00). Hoca bunlara hem kodlama hem sıkıştırma dediğini, nedenini Braille üzerinden açıklayacağını söyledi.

11.1 Braille Kodlama

Braille, görme engelliler için geliştirilmiş 6 noktalı bir alfabedir. Her şey altı noktayla kodlanır; noktaları 3x2'lik bir matris gibi düşünmek gerekir. Örneğin A harfinde birinci satırın birinci kolonunda kabartı vardır; okuyan kişi eliyle izlediğinde bunun A olduğunu anlar.

6 nokta, yani 6 bit varsa $2^6 = 64$ farklı kod üretilebilir. Ama elde 64 karakter yoktur; diyelim 29 karakter vardır. Büyük harfler ayrıca sayılsa 58 olur, noktalama işaretleri de konur; yine de boşta kodlar kalır. Bu kodlara o dilde en çok geçen karakter toplulukları atanır. Slaytta İngilizce için `and`, `for`, `of`, `the`, `with`, `ch`, `gh`, `sh`, `th` grupları vardır. Böylece 4 bayt olan "with" 6 bitle kodlanabilir. Hocanın buna hem kodlama hem sıkıştırma demesinin nedeni budur. Türkçe olsaydı belki en çok kullanılan "lar", "ler" ekleri ya da "için", "ve", "veya" gibi kelimeler bu kodlara verilebilirdi.

Braille'in çeşitleri de vardır. Standart bir alfabe vardır; görme engelli müzisyenler için notaları ve müzik terminolojisini içeren bir alfabe vardır. Programlama için read, write, for gibi ifadeleri sembolize eden kodlar vardır. Matematik için bir kod vardır (derste "Nimet kod" diye geçti; hoca dersin sonunda bir öğrencinin yazdığı açıklamaya dayanarak bunun matematik için olduğunu doğruladı). Edebiyat metinleri için "literary" kod vardır: paragraf, bölüm başı, sayfa sonu gibi şeyleri gösteren noktalar bulunur. Örneğin roman okurken bir bölümün bitip ötekinin başladığını anlatan bir kod vardır.

Ek bilgi (derste anlatılmadı): Derste "Nimet kod" diye geçen matematik Braille'i büyük olasılıkla Nemeth Braille Code'dur. İngilizce Braille'de sık geçen kelime ve harf gruplarının tek hücreyle yazıldığı kısaltmalı yazım "contracted Braille" olarak bilinir.

11.2 Morse Kodlama

Morse kodlaması, yardım çağrısında kullanılan SOS ile bilinen bir kodlamadır. Karakterler ve özel semboller kısa ve uzun çizgilerden oluşur. Hoca, kodlamanın ilk çıktığında yalnız uzun çizgilerden oluştuğunu, daha sonra daha kolay ve anlaşılır olması için kısa ve uzun çizgilerden oluşturulduğunu söyledi.

Zamanlama kuralları:

- Her nokta bir birim, her çizgi üç birimdir. Yani tek vuruş bir birimdir; bir çizgi arka arkaya üç noktalık süreye karşılık gelir.
- Nokta ve çizgilerin arası bir birimdir.
- Karakterler arası üç, kelimeler arası altı birimdir.
- Bir hata yapıldıysa ve son kelimenin silinmesi isteniyorsa karşı tarafa sekiz nokta gönderilir. Bu, "gönderdiğim son kelimeyi sil, hatalıydı, yeniden gönderiyorum" demektir.

  Ders sonrası not: Hoca kelimeler arası boşluğu 6 birim dedi; uluslararası Morse kodunda kelimeler arası boşluk büyük olasılıkla 7 birimdir.

Hoca, bir metin verildiğinde onu hem sesli hem yazılı olarak Morse alfabesine çeviren uygulamalar bulunabileceğini söyledi.

12. Geri Dönüşümsüz Metin Sıkıştırma ve Sparse String

12.1 Geri Dönüşümsüz Metin Sıkıştırma

Sezgisel yöntemlerde geri dönüşümü olmayan sıkıştırma yöntemleri de vardır (irreversible text compression). Örneğin bir veriyi sıkıştırarak göndermek istiyorum; içindeki boşlukları sildim, bütün büyük harfleri küçük harfe çevirdim ya da tam tersini yaptım. Verinin taşıdığı bilgide değişiklik yoktur, yalnız formatında değişiklik vardır. Orijinale dönülemez ama bilgi olarak bir şey kaybedilmez. Bu tür sıkıştırmalar ad hoc sıkıştırma olarak geçer: kişiye ya da veriye özel geliştirilmiş yöntemlerdir.

12.2 Sparse String Sıkıştırma

Ad hoc yöntemler arasında geri dönüştürülebilen basit algoritmalar da vardır; bunun en güzel örneği sparse string sıkıştırmadır. String'in ne olduğu bilinir; sparse veri ise çoğunluğu sıfırlardan oluşan veridir. İçinde çok fazla sıfır olan string'ler farklı ad hoc yöntemlerle sıkıştırılabilir.

Slayttaki örnek (ekrandan, 02:00:00): "Bu dersin adı veri sıkıştırma ve pazartesi günü yapılıyor" metninde kelimeler arasında farklı sayıda boşluk vardır (slaytta boşluklar renkli gösterilmiştir). Aşağıda _ boşluğu gösterir:

```
Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__yapılıyor
```

L1'in oluşturulması: Metinde boşluk mu çok, karakter mi? Karakter. Sparse veri çoğunluğu 0 olan veri olduğu için her karakter yerine 0, her boşluk yerine 1 yazılır. "Bu" için iki 0, ardından iki boşluk için iki 1; "dersin" için altı 0, iki boşluk için iki 1; "adı" için üç 0, üç boşluk için üç 1; "veri" için dört 0, bir boşluk için bir 1 ve böyle devam eder. Metin 64 karakterdir ve 4'er bitlik 16 gruba ayrılır:

```
L1 = 0011|0000|0011|0001|1100|0010|0000|0000|0111|0010|0000|0000|1000|0110|0000|0000
```

L2'nin oluşturulması: L1'deki her dörtlüye bakılır. Dördü de 0 ise 0, sıfırdan farklıysa 1 yazılır:

```
L2 = 1011|1100|1100|1100
```

L3'ün oluşturulması: Aynı işlem L2'ye uygulanır; dört grubun hepsi sıfırdan farklı olduğu için:

```
L3 = 1111
```

L1, L2 ve L3 python ile yeniden hesaplanmış ve slayttakiyle aynı çıkmıştır.

Gönderim: Slayttaki kural: “L1 ve L2’deki sıfır olan gruplar silinerek L1, L2 ve L3 string’leri ve arkasından boşluksuz olarak metin gönderilir.” L1’deki `0000` grupları çıkarılır; L2’de hepsi sıfır olan grup yoktur, L3’te de yoktur:

```
L1 = 0011|0011|0001|1100|0010|0111|0010|1000|0110
L2 = 1011|1100|1100|1100
L3 = 1111
Metin: Budersinadıverisıkıştırmavepazartesigünüyapılıyor
```

Açma: Alıcı L3’ten başlar. L3’ün birinci biti 1 olduğu için L2’nin ilk dörtlüsünü alır; ikinci biti 1 olduğu için ikinci dörtlüsünü, üçüncü ve dördüncüsü de 1 olduğu için üçüncü ve dördüncü dörtlüyü alır. Böylece L2 kurulur. Sonra L2’den L1’e geçilir: L2’nin ilk biti 1 ise L1’deki sıradaki dörtlü alınır, 0 ise dört tane 0 konur (çünkü L1 oluşturulurken bunlar atılmıştı). Bu şekilde ilerleyerek orijinal L1 elde edilir. Son olarak L1’de 0 karakter, 1 boşluk demek olduğu için metin okunur: iki karakter “Bu”, iki boşluk; altı karakter “dersin”, iki boşluk; üç karakter “adı”, üç boşluk; dört karakter “veri”, bir boşluk ve böyle devam ederek orijinal veri boşluklarıyla birlikte elde edilir.

Hoca bunun çok zahmetli ve çok maliyetli olduğunu, çok daha hızlı ve iyi performanslı yöntemler görüleceğini, ama işin en başında olunduğunu ve insanların bu algoritmaları böyle böyle geliştirdiğini söyledi. Örneğin kelimeler arasında standart olmayan boşluklar ve bir sürü boş satır içeren bir belgede bütün boşluklar kaldırılırsa orijinal hâle aynı boşluklarla dönülemez; bu ihtiyacı karşılayacak bir algoritma geliştirilmesi gerekir.

Hoca ad hoc yaklaşımı genelledi: çok matematiksel ya da içinde çok fazla boşluk veya sıfır geçen bir veri setinde var olan yöntemler iyi sonuç vermeyebilir; o zaman veriye özel bir uygulama geliştirilir. İçinde tablo, resim ve normal metin olan bir belge tek bir yöntemle sıkıştırılırsa başarılı olunmaz. Belki segmentasyon yapılır: tablo bir yöntemle, resim başka bir yöntemle, metin başka bir yöntemle sıkıştırılır ve bunların belge içindeki yer bilgileri saklanır; açarken tersi yapılır. Bunların hepsi ihtiyaca göre ortaya atılmış ad hoc uygulamalardır.

13. Baudot Kodlama

Hoca bunu çok basit ama “çok zekice” bir yöntem olarak tanıttı (ekrandan, 02:10:00). Baudot (hocanın deyişiyle yöntemin adı mucidinin soyadıdır), A karakterini de 1 karakterini de aynı kodla kodlar. Normalde A da 1 de 8 bitle kodlanır; burada ikisi de 5 bitle ve aynı bitlerle kodlanır.

Bütün kodlar 5 bitliktir, dolayısıyla $2^5 = 32$ farklı kod vardır. Slayttaki tabloda her kodun bir “Letters” (harf) ve bir “Figures” (rakam/işaret) karşılığı vardır. Örneğin `10000` hem A hem 1’dir; `00110` hem B hem 8’dir. Rakam tarafında 0-9, parantez, yüzde, eşittir, artı, virgül, nokta, iki nokta, eğik çizgi gibi işaretler vardır.

Slayttaki tablonun bir kısmı:

Harf	Kod	Rakam/işaret	Harf	Kod	Rakam/işaret
A	10000	1	Q	10111	/
B	00110	8	R	00111	-
C	10110	9	S	00101	SP
D	11110	0	T	10101	na
E	01000	2	U	10100	4
H	11010	+	Y	00100	3
K	10011	(	LS	00001	LS
L	11011	=	FS	00010	FS
O	11100	5	CR	11000	CR
P	11111	%	LF	10001	LF

Kısaltmalar: LS, Letter Shift; FS, Figure Shift; CR, Carriage Return; LF, Line Feed; ER, Error (`00011`); na, Not Assigned (atanmamış); SP, Space.

Çözücü bir kodun A mı 1 mi olduğunu nasıl bilir? Burada LS ve FS kodları devreye girer. Örneğin bir kod hem B hem 8’e karşılık geliyorsa, bu kod yazılmadan önce “letter shift” yazılır:

“bundan sonra okuyacağın kodları harf olarak oku”. Rakama geçerken de “figure shift” kullanılır.

Atanmamış (na) kodlar kullanıcıya seçimlik bırakılmıştır. Örneğin bir özel sembol kullanılmak istenirse (hocanın örneğiyle “süslü a”) boştaki bir koda atanabilir. Tabloda küçük-büyük harf ayrımı yoktur; ama boştaki kodlardan biri “bundan sonraki karakter büyük harf” anlamında, biri küçük harf anlamında kullanılarak büyük-küçük harf geçişleri de sağlanabilir.

Hocanın değerlendirmesi: bütün semboller otomatik olarak 5 bitliktir. LS ve FS için de 5 bit harcanır, ama metinde bir sayı bir karakter diye sürekli değişim olmaz; genelde sayılar arka arkaya ya da karakterler arka arkaya gelir. Belirli bir sayıda şey yazıldıktan sonra geçiş yapmak gerekir. Hoca, tamamen karakterlerden oluşan, tamamen matematiksel ya da karışık bir belgede bu kodlamanın başarı oranının nasıl değiştiğinin incelenebileceğini söyledi.

14. Önek Sıkıştırma (Front Compression)

Hoca bunu “çok tatlı bir sıkıştırma yöntemi” olarak tanıttı (ekrandan, 02:15:00). Elde açıklamaları olmayan bir yazım kılavuzu olsun ve sıkıştırılarak gönderilmek istensin. Kelimeler alfabetik sıradadır: önce birinci karaktere, sonra ikinci, üçüncü karaktere göre sıralanır.

Yöntem: her kelimenin önüne, kendinden önce gelen kelimeyle soldan ortak olan karakter sayısı yazılır; ardından kalan karakterler yazılır. Slayttaki liste:

Kelime	Kodu	Kelime	Kodu
a	a	abash	3sh
aardvark	1ardvark	abated	3ted
aback	1back	abate	3te
abaft	3ft	abbot	2bot
abandon	3ndon	abbey	3ey
abandoning	3ndoning	abbreviating	3reviating
abasement	3sement	abbreviate	9e
abandonment	3ndonment	abbreviation	9ion

Okuma: `1ardvark`’ın önünde 1 karakter vardır; bu, kendinden önceki kelimenin soldan ilk karakteridir, yani “a”. `1back` için de önceki kelimenin ilk karakteri “a” alınır: “aback”. `3ft` için önceki kelimenin (aback) ilk üç karakteri “aba” alınır: “abaft”.

Kazanç: Başlarda çok kazanç getirmiyormuş gibi görünse de soldan tekrar eden karakterler arttıkça kazanç artar. “abandoning” 10 karakterdir; karakter başına 8 bitle 80 bit eder. Bu algoritmayla `3ndoning` 8 karakterdir, yani 64 bit. “abbey” 5 karakterle 40 bit iken `3ey` 3 karakterle 24 bittir.

Kelimeler karışık sırada olsaydı böyle bir sıkıştırmadan söz edilemezdi; performans alınamaz, hatta veri genişlerdi.

15. Run-Length Encoding (RLE)

Hoca bugünkü son yöntemin Run-Length Encoding olduğunu, genelde kimsenin uzun adını söylemediğini, RLE dendiğini belirtti.

> **Hocanın vurgusu:** RLE’yi “yöntem deyip geçmeyin”. Kullandığımız birçok algoritmada RLE vardır: JPEG resimleri sıkıştıran algoritmanın içinde, WinRAR’ın ve WinZip’in içinde ve başka algoritmaların bir aşamasında RLE kullanılır.

15.1 Tanım

Slayttaki tanım (ekrandan, 02:20:00):

- Bir input stream içinde bir “d” verisi “n” defa arka arkaya geliyorsa bu bir seferde “nd” çifti ile gösterilebilir.
- “n” değerine “run” denir.

- Hem metin hem görüntü sıkıştırmada kullanılır.
- Kayıplı ve kayıpsız versiyonları vardır.
- Run-Length Encoding ya da RLE olarak adlandırılır (slaytta “Run Lenght”).

Hoca stream’in akış demek olduğunu hatırlattı: girdi akış hâlinde gelir ve bayt bayt, bit bit ya da 8, 16, 128 bitlik bloklar hâlinde alınabilir. RLE’nin en ilkel hâlinde biçim şudur: önce run değeri (tekrar sayısı), ardından karakterin kendisi. Örneğin `aaa` → `3a`.

15.2 Metin mi Görüntü mü?

Hoca “RLE metinde mi görüntüde mi daha başarılıdır?” diye sordu.

Soru (hocanın sorusu): RLE hangisinde daha başarılıdır? **Cevap (bir öğrenci):** Resimlerde daha başarılıdır; resim bir matris biçimindedir ve bitsel olarak bakıldığında aynı renkler aynı kodlarla gösterilir. Örneğin siyah bir alan birkaç karede devam eder, yani arka arkaya gelen aynı değer daha çoktur.

Hoca cevabı onayladı: bir metinde arka arkaya aynı karakterin gelmesi, Türkçede “saat” gibi kelimeler dışında çok zordur; belki birkaç sessiz harf arka arkaya gelir ama çok azdır. Metinde ancak boşluklar RLE ile kodlanabilir; onun dışında iyi performans elde edilemez. Ama siyah-beyaz bir resim 0 ve 1’lerden oluşur ve 0’lar ve 1’ler ister istemez sıra sıra devam eder. Renkli bir resimde bile, örneğin siyah bir mousepad görüntüsünde arka arkaya gelen bir sürü siyah kodu olur; orada RLE elbette daha başarılıdır.

15.3 Kayıplı RLE

RLE hem kayıplı hem kayıpsız kullanılabilir. Hocanın örneği: bir resimde piksel değerleri `20, 20, 20, 20, 21, 20, 20, 20, 20` olsun; 9 piksel vardır. 20 diyelim ki kırmızının kodudur; 21 de kırmızıya yakın, insan gözünün ayırt edemeyeceği bir renktir. Renk skalasında art arda gelen bu iki rengi göz ayırt edemeyeceği için 21 yerine 20 konursa daha iyi bir sıkıştırma yapılır. Ama açıldığında 21 bulunamaz; bu kayıplı bir sıkıştırmadır.

15.4 RLE Örneği ve Negatif Sıkıştırma

Slayttaki örnek (ekrandan, 02:25:30): gerçek dünyada böyle bir girdi olmaz ama olduğunu düşünelim:

```
Uncompressed: aaaaabbbbbbbbbbbccccddddddddeeeeeeeeee
Compressed:   5a12b4c9d10e
```

5 tane a, 12 tane b, 4 tane c, 9 tane d, 10 tane e vardır. Okunan run değeri 12 ise bu “1, 2” diye değil, 12 tane b diye okunur.

Hoca burada bir soru açtı: orijinal metinde de rakamlar olabilirdi (örneğin 8 ya da 88 gibi bir sayı); bunun metinde geçmediği nereden bilinecek ve run değeriyle nasıl karıştırılmayacak? Bu konuya 15.5’te dönüldü; önce negatif sıkıştırma anlatıldı.

Negatif sıkıştırma:

```
Uncompressed: aabccdeefghijjjklmnopqrrstttuvvwwxyyyz
Compressed:   2a1b2c1d2e1f1g1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z
```

Orijinal 38 bayttır. RLE ile kodlanınca hoca 50 bayt saydı: 38 baytlık veri sıkıştırılmak istenirken 50 bayta çevrildi. Nedeni, tek geçen her karakter için 2 bayt harcanmasıdır: bir tane b için `1b`, bir tane d için `1d`, bir g için `1g` yazılır. Arka arkaya ikiden fazla gelen karakter sayısı tek tek gelenlerden azsa hiçbir zaman sıkıştırma yapılamaz; negatif sıkıştırma olur, çıktı orijinalden büyük olur. Hoca buna sıkıştırma değil genişletme dedi. Metinde iyi sonuç vermemesinin nedeni de budur.

> **Ders sonrası not:** Slayttaki sıkıştırılmış dizide `1h` eksik yazılmıştır (`1g1i`). Orijinalde h bulunduğu için tam dizi `2a1b2c1d2e1f1g1h1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z` olmalıdır; bu 52 bayttır (python ile doğrulandı). Hocanın saydığı 50 bayt, slayttaki eksik diziye karşılık gelir (eksik dizi python ile sayıldığında 50 bayttır). Sonuç değişmez: veri genişler.

15.5 Rakamlarla Karışma ve Escape Kodu

Slayttaki örnek (ekrandan, 02:30:00): “2. all is too well” (bir listedeki ikinci madde) sıkıştırılsın.

```
Uncompressed: 2. all is too well
Compressed:   2. a2l is t2o we2l
```

Baştaki “2” orijinaldir, sıkıştırma değildir; nokta ve a yazılır; iki tane l için run değeri 2 ve l yazılır; i, s, t tek; iki o için `2o`; iki l için yine `2l`. Slayttaki sorular: 1) Sıkıştırma var mı? 2) Decompressor 2’leri anlayacak mı?

Karşı taraf bunu açabilir mi? Baştaki “2”nin ikinci madde değil de iki tane nokta olarak algılanmayacağının garantisi yoktur. “İlk okuduğum run, ikincisi karakterin kendisi” kuralı her karaktere uygulanırsa (her karakterin önüne run yazılırsa) metin çok genişler. Bir yol genişletir, öteki yol sıkıştırmaz; RLE’nin önüne her zaman sıkıştırmaya tam uyan girdiler gelmeyebilir.

Çözüm: özel escape kodu. Özel bir sembol kullanılır; o sembolden sonra gelen değer run değeri olarak okunur. Slayttaki örnekte metinde hiç geçmeyen `@` seçilmiştir:

```
Compressed: 2. a@2l is t@2o we@2l
```

Okuma: “2.” orijinaldir; a; `@` özel sembol, ardından gelen 2 run değeri, ardından tekrar eden karakter l, yani iki l; i, s; t; `@2o` iki o; w, e; `@2l` iki l.

Soru: Özel karakteri nasıl belirleyeceğim? **Cevap:** Önce belgenin içine bakılır; belgede geçmeyen bir karakter varsa o özel karakter yapılır. Normal bir metinde 0’dan 255’e kadar bütün karakterlerin geçmediği bilinir.

Soru: Ya bir exe dosyası sıkıştırılıyorsa? **Cevap:** O zaman bu yapılamaz. Exe dosyasının içinde 0’dan 255’e kadar bütün sayılar olabilir; hiç kullanılmayan bir sembol bulunamayabilir. Bu durumda başka bir yol izlenir. Hepsinin özünde RLE vardır, ama uygulamanın içeriğine ve eldeki veriye göre farklı versiyonlar geliştirilmiştir.

15.6 RLE Ne Zaman Kazanç Sağlar?

- Bir tane a RLE ile `1a` diye yazılırsa 1 bayt 2 bayt olur.
- İki tane a `2a` diye yazılırsa 2 bayt 2 bayt kalır; sıkıştırma yoktur.
- Üç tane a `3a` diye yazılırsa 3 bayt 2 bayta iner.

Bu yüzden RLE 3 ve daha fazla tekrar eden karakterlerde kullanılır. İki kez tekrarlanan bir karakterde RLE’ye gerek yoktur; orijinal hâli kullanılır, çünkü sıkıştırmaya katkı sağlamaz.

16. RLE’nin Temel Problemleri ve MNP5

Slayttaki “Temel problemler” (ekrandan, 02:37:20):

- Doğal dilde yazılmış metinlerde arka arkaya tekrar eden karakter, boşluklar hariç, nadirdir.
- Tekrar sayısına 1 bayt ayrılır ve 255 ile sınırlıdır. Tekrar sayısı 3 tekrar karakterden sonra kullanılacaksa 258 adet aynı karakter 3 bayt ile kodlanır.
- Kullanılan özel escape kodu metin içinde geçebilir; farklı bir karakter seçilmelidir.
- Exe türü dosyalarda tüm ASCII karakterler dosya içinde yer alabilir. Bu durumda farklı bir yol izlenmelidir.

Run değeri 1 bayta yazıldığı için en çok 255 olabilir. Hocanın örneği: 257 tane a arka arkaya gelmişse 257 run değerine sığmaz. Önce `255a` yazılır (run 255, karakter a), kalan iki a da olduğu gibi yazılır; toplam 257 olur.

> **Ek bilgi (derste anlatılmadı):** Slayttaki 258 büyük olasılıkla şu düşünceden gelir: run yalnız 3 ve üstü tekrarlarda kullanıldığı için run baytına tekrar sayısının kendisi değil, 3 eksiği yazılabilir. O zaman 1 baytlık 0-255 aralığı 3-258 tekrarı gösterir ve 258 aynı karakter “escape + run + karakter” biçiminde 3 baytla kodlanır.

16.1 MNP5

Exe dosyası sıkıştırılıyorsa ve bütün karakterler kullanılmışsa başka bir yöntem çıkar; arka planında yine RLE vardır. Bu, Microcom Network Protocol, yani MNP5 yöntemidir (MNP7 de vardır, yeri gelince anlatılacaktır). Ağlarda kullanılan ve Microcom firması tarafından ortaya atılmış bir protokoldür.

Slayttaki kural (ekrandan, 02:37:20): input stream'de 3 karakter arka arkaya tekrar ediyorsa output stream'e 3 kopya yazılır; dördüncü bayta run değeri yazılır.

```
3 kopya için (aaa)    -> aaa0 -> 3 bayt yerine 4 bayt (genişleme)
4 kopya için (aaaa)   -> aaa1 -> 4 bayt yerine 4 bayt (sıkıştırma yok)
5 kopya için (aaaaa)  -> aaa2 -> 5 bayt yerine 4 bayt (sıkıştırma var)
```

Üç karakter varsa run değeri 0'dır ve 3 karakter için 4 bayt yazılmış olur; bu bir genişlemedir. Dört a varsa fazlalık (1) run değerine konur; 4 bayt 4 bayt olur. Beş a varsa üç tanesi yazılır, iki tanesi run olarak konur; 5 bayt 4 bayta iner ve sıkıştırma elde edilir. Run'lar sona gelir.

Önceki RLE versiyonlarında 3 ve daha fazla tekrarda run değeri verilerek yazılıyordu, aksi hâlde karakterler olduğu gibi yazılıyor ve run hiç konmuyordu. MNP5 ise bir karakter 3 kez arka arkaya geldiyse, ardından gerçekten tekrar gelsin ya da gelmesin, mutlaka bir run baytı yazılmasını ister.

Hocanın örneği (02:36:57): `b a c c d d d 0 f d d d 6` gibi bir dizide çözücü bir sayının karakter mi run mı olduğunu şöyle anlar: b tek, a tek; iki tane c var, iki tekrar run gerektirmediği için arkasından run gelmez. Üç tane d gelince arkasından mutlaka bir run gelir; hiç ek tekrar yoksa bu 0'dır (hoca önce 0'ı yazmayı unuttu, sonra düzeltti). Sonra f; sonra yine üç d ve run değeri 6: bu, $3 + 6 = 9$ tane d demektir.

Hocanın özeti: sonuçta yine "run, karakter" biçimi kullanılır, ama farklı yöntemlerin içinde farklı şekillerde kullanılır. Escape kodlu RLE'de özel sembol, ardından run, ardından karakter gelir (`@3a`). MNP5'te arka arkaya 3 tane a gelmişse arkasından mutlaka run gelir ve 4 bayt kullanılır (`aaan`).

16.2 RLE ve MNP5 Performans Karşılaştırması

Performans, önceki derste görüldüğü gibi output stream bölü input stream ile ölçülür. Slayttaki tanımlar (ekrandan, 02:40:00): N karakter sayısı, M tekrar eden grup sayısı, L gruplardaki ortalama eleman sayısı.

$$\text{RLE performansı} = \frac{N - (M \cdot L) + M \cdot 3}{N}, \qquad \text{MNP5 performansı} = \frac{N - (M \cdot L) + M \cdot 4}{N}$$

Hocanın açıklaması: input N karakterdir. Output'u bulmak için her grubun L elemanı N'den çıkarılır; geri kalanlar tekil kullanılmış karakterlerdir. Escape kodlu RLE'de her grup 3 baytla (özel sembol, run, karakter) kodlandığı için $3M$ eklenir. MNP5'te her grup 4 baytla (3 kopya + run) kodlandığı için $4M$ eklenir.

Escape kodlu RLE'yi kullanmak biraz risklidir, çünkü exe dosyalarında özel sembol yerine geçecek bir karakter bulunmayabilir. MNP5'te böyle bir problem yoktur.

Örnek "Örnek Sorular" bölümünde çözülmüştür: $N = 1000$, $M = 10$, $L = 5$ için RLE %2, MNP5 %1 sıkıştırır.

Hoca, özel sembol bulunabiliyorsa RLE'yi seçmenin daha iyi olacağını, ama her zaman özel sembol bulunamayabileceğini söyledi. Aralarında çok büyük bir fark olmadığı için MNP5 kullanılabilir. Ağda gönderilen dosyaların büyük çoğunluğu exe ya da sıkıştırılmış dosyalardır; bunlar bütün karakterleri içerebildiği için MNP5 ile kodlanarak gönderilir.

Örnek Sorular

Soru 1 (ekrandan, 00:30:00): Her iki bit için `00 → 00001`, `01 → 01010`, `10 → 10100`, `11 → 11111` kod kelimeleri kullanılıyor. Orijinal mesaj `10 11 01 00 11`'dir. Alıcıya `00100 11111 00010 00000 11111` gelirse orijinal mesaj nasıl bulunur?

Çözüm (derste verildi, 00:31:13-00:33:54):

- Gönderilen: `10100 11111 01010 00001 11111`.
- `00100`: tabloda böyle bir kod yok, yani bozulma var. Hangi kodun bir bit bozulmuş hâli `00100`'ü verir? `10100`'ün birinci biti bozulmuştur → `10`.
- `11111`: tabloda var, bozulma yok → `11`.
- `00010`: tabloda yok. `01010`'un ikinci biti bozulmuştur → `01`.
- `00000`: tabloda yok. `00001`'in beşinci biti bozulmuştur → `00`.
- `11111`: bozulma yok → `11`.
- Sonuç `10 11 01 00 11`; gönderilen mesajın aynısıdır. Tek bitlik bozulmalar anında tespit edilip düzeltildi.

Soru 2 (ekrandan, 00:40:00; derste sözlü, 00:40:32): Mesaj `1100` Hamming koduyla nasıl kodlanır?

Çözüm (derste verildi): $b_5 = 1 \oplus 1 \oplus 0 = 0$, $b_6 = 1 \oplus 0 \oplus 0 = 1$, $b_7 = 1 \oplus 0 \oplus 0 = 1$. Kod kelimesi `1100011`.

Soru 3 (ekrandan, 00:46:30): Hamming kodlamada alıcıya `1100100` geliyor. Bozulma var mı, varsa hangi bit bozulmuştur ve doğru mesaj nedir?

Çözüm (kontroller derste verildi, sonrası 00:53:03'teki genel kuraldan kuruldu):

- $P_1 = b_1 \oplus b_2 \oplus b_3 \oplus b_5 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$
- $P_2 = b_1 \oplus b_3 \oplus b_4 \oplus b_6 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$
- $P_3 = b_2 \oplus b_3 \oplus b_4 \oplus b_7 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$
- Üç kontrol de 1; bozulma vardır. Üçünde de bulunan bit b_3'tür. (ara adım eklendi) b_3 ters çevrilince `1110100` olur; yeni kontroller $1 \oplus 1 \oplus 1 \oplus 1 = 0$, $1 \oplus 1 \oplus 0 \oplus 0 = 0$, $1 \oplus 1 \oplus 0 \oplus 0 = 0$ verir. Mesaj `1110`'dur (tablodaki `1110 → 1110100` satırı).

Soru 4 (ekrandan, 00:50:00): Hamming çözmede $P_3 = 0$, $P_1 = P_2 = 1$ bulunursa ne yapılır?

Çözüm (derste verildi): P_1 ve P_2'de olup P_3'te olmayan tek bit b_1'dir. $\tilde{b}_1$ bitinin tersi alınarak doğru kod kelimesi elde edilir.

Soru 5 (ekrandan, 00:56:30): Veri `0100010100` için tek eşlik ve çift eşlik yöntemlerinde eşlik biti ne olur?

Çözüm (derste verildi): Birlerin sayısı 3'tür (tek). Tek eşlikte eşlik biti 0 (`01000101000`), çift eşlikte eşlik biti 1 (`01000101001`) olur.

Soru 6 (ekrandan, 01:20:00): UPC kodu `0 48500 00139` olan ürünün kontrol dijiti nedir?

Çözüm (derste verildi): $3 \cdot 0 + 4 + 3 \cdot 8 + 5 + 3 \cdot 0 + 0 + 3 \cdot 0 + 0 + 3 \cdot 1 + 3 + 3 \cdot 9 = 0 + 4 + 24 + 5 + 0 + 0 + 0 + 0 + 3 + 3 + 27 = 66$. $A = 66 \bmod 10 = 6$, $C = a_{12} = 10 - 6 = 4$. Kontrol dijiti 4'tür ve barkodda bulunur ama rakamla yazılmaz (python ile doğrulandı).

Soru 7 (ekrandan, 01:25:40): Posta barkodunda okunan sayı `2007?55761` ve beşinci dijit okunamadı (son 1 kontrol dijiti). Okunamayan dijit nedir? Oluşan bir dijitlik hata düzeltilebilir mi?

Çözüm (derste verildi, 01:26:13-01:27:05): Bilinen dijitlerin toplamı $2+0+0+7+5+5+7+6+1 = 33$'tür; slayttaki ifade $(33 + ?) \bmod 10$'dur. Toplamın 10'a tam bölünmesi için $? = 7$ olmalıdır: $33 + 7 = 40$, $40 \bmod 10 = 0$. Yani hatalı ya da okunamayan tek dijit düzeltilebilir; sayı `2007755761`'dir (python ile doğrulandı).

Soru 8 (ekrandan, 01:34:50): Gönderilecek mesaj `0111101`, üreteç fonksiyonu $G(x) = x^3 + x^2 + 1$. CRC kodu nedir?

Çözüm (slaytta verildi): Mesajın arkasına `000` eklenir, `1101` ile adım adım XOR'lanır (işlem satırları Bölüm 9.1'de). Slayttaki sonuç CRC = `111`, gönderilen `0111101111`.

> **Çözüm (derste verilmedi):** Standart CRC'de (baştaki bit 0 iken `0000` ile XOR) `0111101000`'in `1101`'e bölümünden kalan `011`'dir; gönderilen `0111101011` olur ve alıcıda kalan `000` çıkar. Fark, slayttaki ilk adımda baştaki 0 bitine rağmen `1101` ile XOR yapılmasından gelir (python ile doğrulandı).

Soru 9 (ekrandan, 01:40:00): Encode edilmiş mesaj `0111101111` hatalı mı gitti, yoksa doğru mu? Üçüncü bit hatalı alınmış olsaydı anlaşılacak mıydı?

Çözüm (slaytta verildi, slayttaki işlem yoluyla): `0111101111` için sonuç `0000` → bozulma yok. Üçüncü bit hatalıyken alınan `0101101111` için sonuç `0001` → sıfır değil, bozulma var; ama yeri bulunamaz, veri yeniden istenir (python ile slayttaki yol yeniden yapıldı, aynı sonuçlar çıktı).

Soru 10 (ekrandan, 02:00:00): `Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__yapılıyor` metni (boşluk yerine 1) sparse string yöntemiyle nasıl gönderilir?

Çözüm (derste verildi): L1 = `0011|0000|0011|0001|1100|0010|0000|0000|0111|0010|0000|0000|1000|0110`, L2 = `1011|1100|1100|1100`, L3 = `1111`. Sıfır grupları silinmiş L1 = `0011|0011|0001|1100|0010|0111|0010|10`, L2, L3 ve boşluksuz metin gönderilir (python ile doğrulandı).

Soru 11 (derste sözlü, 02:19:54): RLE metinde mi, görüntüde mi daha başarılıdır? Neden?

Çözüm (derste verildi): Görüntüde. Görüntüde aynı renk değerleri (örneğin siyah-beyaz resimde 0 ve 1'ler, siyah bir alan) arka arkaya çok gelir. Doğal dildeki metinde boşluklar dışında arka arkaya tekrar eden karakter nadirdir.

Soru 12 (ekrandan, 02:25:30): `aabccdeefghijjjklmnopqrrstttuvvwwxyyyz` RLE ile kodlanırsa sıkıştırma olur mu?

Çözüm (derste verildi): Hayır; 38 baytlık veri genişler (negatif sıkıştırma), çünkü tek geçen her karakter için 2 bayt yazılır. Hoca slayttaki diziyle 50 bayt saydı; `1h` dahil tam dizi 52 bayttır (python ile doğrulandı).

Soru 13 (ekrandan, 02:30:00): "2. all is too well" metni `2. a2l is t2o we2l` olarak sıkıştırılırsa sıkıştırma var mı, decompressor 2'leri anlayacak mı? Nasıl çözülür?

Çözüm (derste verildi): Çözücü baştaki "2"nin orijinal rakam mı run mı olduğunu ayırt edemez. Metinde geçmeyen bir escape karakteri (`@`) seçilir ve yalnız ondan sonraki sayı run olarak okunur: `2. a@2l is t@2o we@2l`.

> **Çözüm (derste verilmedi):** Sıkıştırma sorusunun sayısal cevabı: orijinal "2. all is too well" 18 karakterdir; `2. a2l is t2o we2l` de 18 karakterdir, yani iki karakterlik tekrarlarda kazanç yoktur. Escape'li `2. a@2l is t@2o we@2l` ise 21 karakterdir, yani genişler (python ile sayıldı). Bu, derste söylenen "RLE 3 ve üstü tekrarda kullanılır" kuralıyla uyumludur.

Soru 14 (ekrandan, 02:40:00): $N = 1000$, $M = 10$, $L = 5$ için RLE ve MNP5 sıkıştırma performansı nedir?

Çözüm (slaytta verildi): RLE $= (1000 - 50 + 30)/1000 = 0{,}98$ (%2 sıkıştı); MNP5 $= (1000 - 50 + 40)/1000 = 0{,}99$ (%1 sıkıştı).

Dersin Toparlaması

Dersin ilk yarısında veri iletiminde oluşan hataların sezilmesi ve düzeltilmesi işlendi. Hatayı sezmenin düzeltmekten kolay olduğu, tek bitlik (spike) hataların düzeltilebildiği, çok bitlik (burst) hatalarda ise verinin yeniden istendiği vurgulandı. İleri yönlü hata denetimi (FEC) ile otomatik tekrar isteği (ARQ) karşılaştırıldı. Tekrar kodlarından (1 bit için 2 ve 3 bit) başlayıp 2 bit için 5 bitlik koda (2,5 bit) ve Hamming koduna (1,75 bit) gelinerek, tek bit hatayı düzeltirken eklenen bit sayısının nasıl azaltılabileceği gösterildi. Eşlik biti, UPC ve posta barkodlarındaki kontrol dijitleri, CRC ve özet fonksiyonu olarak MD5 ile konu günlük hayata bağlandı.

İkinci yarıda sıkıştırmanın temel tekniklerine geçildi: Braille ve Morse gibi sezgisel kodlamalar, geri dönüşümsüz metin sıkıştırma, sparse string, Baudot, önek sıkıştırma ve RLE. RLE'nin negatif sıkıştırma, rakamlarla karışma, 255 run sınırı ve exe dosyalarında escape karakteri bulunamaması gibi sorunları ve MNP5 çözümü işlendi. Hoca bir sonraki derste RLE'nin görüntü sıkıştırmada kullanımıyla devam edeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.